## 2026-4-21 C++类和对象：上


- 1. 类的定义
  - 1.1类定义的形式
  - 1.2访问限定符
    - 1.2.1什么是访问限定符?
    - 1.2.2访问限定符的作用是什么呢?
    - 1.2.3 访问限定符的默认规则?
  - 1.3类域
- 2.实例化
  - 2.1什么是实例化?
  - 2.2对象的大小
- 3.this指针
- 4.C++和C语言的实现Stack对比


## 1. 类的定义

我们说类实际上是C语言结构体衍生出来的，所以说它和结构体还是有很多类似的地方的。

### 1.1类定义的形式

我们先来举个例子：（以栈为例）
从下述代码我们可以得到如下的信息：

1. class是这个类的关键字（比较类似于定义struct),Stack是这个类的一个名称。
2. 关于public、private其实上是访问限定符号。我们只能访问public下的成员函数，但不能直接访问private修饰的成员变量
3. 我们可以将函数放到类域里，C语言中在struct里面只能定义变量，C++中允许定义成员函数。
   **变量在声明的时候是不开辟空间的，但是在定义的时候是开辟空间的。**
4. 定义在类里面的成员函数**编译器可能会按内联函数处理**，但**并不等价于** `inline` 关键字。实际是否内联由编译器决定。

```
#include<iostream>
#include<assert.h>

using namespace std;
class Stack {
```

```
public:
    void StackInit(int n =4) {//成员函数
        array = (int*)malloc(sizeof(int) * n);
        if (nullptr == array) {
            perror("malloc申请空间失败");
            return;
        }
        capacity = n;
        top = 0;
    }
    void Push(int x) {
        array[top++] = x;
    }
    int Top() {
        assert(top > 0);
        return array[top - 1];
    }
    void Destroy() {
        free(array);
        array = nullptr;
        top = capacity = 0;
    }
private://成员变量，一般不让修改
    int* array;//这里并不是变量的定义，这里是变量的声明
    size_t capacity;
    size_t top;
};
int main() {
    Stack st;//实例化一个对象，所以说st实际上就是我们的对象
    st.StackInit();
    st.StackInit();
    st.Push(3);
    cout << st.Top() << endl;
    st.Destroy();
    return 0;
}
```

下面我们来举第二个例子：
为了区分成员变量，一般习惯上会将成员变量加一个特殊的标识
如_或者m开头

```
#include<iostream>
using namespace std;
class Date
```

```
{
public:
    void Init(int year, int month, int day)
    {
        _year = year;
        _month = month;
        _day = day;
    }
private:
    //如下示例
    int _year;
    int _month;
    int _day;
};
int main() {
    Date d;
    d.Init(2024, 3, 31);
    return 0;
}
```

C++中对结构体进行了升级：
对比一下：

```
typedef struct ListNodeC
{
    struct ListNode* next;//在C语言中，我们说这里的struct是不能去掉的
    //因为这个结构体我们还没有定义完成，这个时候还不能使用
    int val;
}LTNode;
```

但是在C++中：我们不再需要typedef,可以让struct对应的名字代表该类型

```
struct ListNodeCPP
{
    void Init(int x)
    {
        next = nullptr;
        val = x;
    }
    ListNodeCPP* next;//可以不要这个struct
    int val;
};
int main() {
```

```
    return;
}
```

### 1.2访问限定符

#### 1.2.1什么是访问限定符?

C++实现封装的一种方式，就是用类将对象的属性和方法结合到一起，让对象更加的完善，通过访问权限选择性的将其接口提供给外部用户使用。C++的访问限定符有：public、protected、private三种

#### 1.2.2访问限定符的作用是什么呢?

public修饰的成员变量可以在类外直接被访问；protected和private修饰的成员变量在类外不能直接被访问，目前看来protected和private的作用是差不多的，后续继承章节详细介绍其作用。
访问权限作用域从该访问限定符开始到下一个访问限定符出现为止，如果我们后面没有访问限定符，作用域就跟随类结束而结束。

#### 1.2.3 访问限定符的默认规则?

class定义成员没有访问限定符符号修饰时候**默认为private**,struct默认为public。
一般情况下，成员变量会被限制为private/protected,需要给别人使用的成员函数会放public。

### 1.3类域

1. 类定义了一个新的作用域，类的所有成员都在类的作用域中，在类外定义成员时，需要使用::作用域操作符来指明成员属于哪个类域。
2. 类域影响的是编译时候的查找规则。举例如下，如果StackInit如果不指定类域Stack,那么编译器就把StackInit当成全局函数，那么编译的时候，**找不到array等成员的声明和定义在哪里，就会报错**。指定类域Stack,就是知道StackInit是成员函数，当前域找不到array等成员，就会到类域中去查找。

```
using namespace std;
class Stack {
public:
void StackInit(int n = 4);
private:
```

```
    int* array;
    size_t capacity;
    size_t top;
};

void Stack::StackInit(int n = 4) {//声明与定义分开，需要指定类域
    array = (int*)malloc(sizeof(int) * n);
    if (nullptr == array) {
        perror("malloc申请空间失败");
        return;
    }
    capacity = n;
    top = 0;
}

int main() {
    Stack st;
    st.StackInit();
    return 0;
}
```

我们来思考一个问题：我们为什么需要类域?
我们来举一个实际的例子：我们学习栈和队列的时候，我们有Push、Pop等相似的方法，当然这些类在不同的数据结构中的实现肯定是不同的，那么如果我们在全局同时有两个Push呢?这个时候就会发生冲突，所以我们说类域的最大的作用在于解决类与类中不同方法的冲突问题。

## 2.实例化

### 2.1什么是实例化?

实例化就是用类型在物理内存中创建对象的过程，称为类实例化对象。

1. 类是对对象的一种抽象化描述，是一个模型一样的东西，限定了类有那些成员变量，这些成员变量只是声明，没有分配空间，用类实例化出对象时才会分配空间。
2. 一个类可以实例化出多个对象，实例化出的对象会占用实际的物理空间，用来存储类的成员变量。类就像设计图纸一样，不能存储数据，实例化出的对象分配物理内存存储数据。

```
using namespace std;

class Date
{
```

```
public:
    void Init(int year, int month, int day)
    {
        _year = year;
        _month = month;
        _day = day;
    }
    void Print() {
        cout << _year << "/" << _month << "/" << _day << endl;
    }
private:
    //这里只是声明，没有开辟空间
    int _year;
    int _month;
    int _day;
};
int main() {
    //实例化出对象d1和d2
    Date d1;
    Date d2;
    d1.Init(2025, 4, 10);
    d1.Print();
    d2.Init(2025, 4, 21);
    d2.Print();

    return 0;
}
```

### 2.2对象的大小

类实例化出每一个对象，都有独立的数据空间，所以对象中肯定包含成员变量（变量是占空间的）那么对象中是否含有成员变量呢?

1. 首先，函数被编译后实际上是一段指令，对象中没有办法存储，这些指令存储在一个单独的区域（代码段），如果对象非要存储函数的话只能存储成员函数的指针。
2. 那我们来分析一下，对象中是否有存储成员函数的必要呢？在上述实例中，Date实例化出d1和d2两个对象，它们有自己独立的成员变量存储各自的年、月、日数据。但是d1和d2的成员函数Init/Print指针确是一样的，实际上不需要存储到对象中。（公用的嘛）
3. 所以，函数指针是不需要存储的，函数指针是一个地址，调用函数被编译成汇编指令[call 地址]，实际上编译器在编译链接时，就要找到函数的地址，不是在运行的时候找的。
   **所以对象中只存储成员变量，C++规定类实例化的对象也要符合内存对齐规则**（这里不过多赘述内存对齐，其本质上时一个空间换时间的方式）
   下面的对象分别占多少个字节呢?

```cpp
using namespace std;

class A
{
public:
    void Print() {
        cout << _ch << endl;
    }
private:
    char _ch;
    int _i;
};
class B
{
public:
    void Print() {

    }
};
class C
{

};
int main() {
    A a;
    B b;
    C c;
    cout << sizeof(a) << endl;
    cout << sizeof(b) << endl;
    cout << sizeof(c) << endl;
    return 0;
}
```

上述程序运行后输出：8 1 1
为什么B和C类的对象是1呢？为啥还要给它一个字节？因为如果一个字节都不给它，怎么能标识它存在过呢？所以这里给一个字节是为了占位标识对象存在。

### 3.this指针

1. Date类中有Init与Print两个成员函数，函数体中没有任何对于对象的区分，那当d1调用Init和Print函数的时候，我们怎么能确定访问的到底是d1的对象还是d2的对象呢？**C++中一个隐含的this指针就解决了这个问题**

2. 编译器在编译后，类的成员函数默认都会在第一个形参的位置添加一个当前类型的指针，叫做this指针。所以说Date中Inite函数的原型为：

```
void Init(Date* const this,int year, int month, int day)
```

3. 类的成员函数中访问成员变量，本质上都是通过this指针访问的，如Init函数给_year赋值就是：this->_year=year;
4. C++规定，不能在实参和形参的位置显示的写this指针（编译时，编译器会处理），但是可以在函数体中显示使用this指针(this指针就相当于把对象的地址传给了成员函数)

```
using namespace std;
class Date
{
public:
    //void Init(Date* const this,int year, int month, int day)
    void Init(int year, int month, int day)
    {
        //this = nullptr;这样是不允许的
        this->_year = year;//实际上也就是这样
        _month = month;
        _day = day;
    }
    void Print() {
        cout << _year << "/" << _month << "/" << _day << endl;
    }
private:
    //这里只是声明，没有开辟空间
    int _year;
    int _month;
    int _day;
};
int main() {
    //实例化出对象d1和d2
    Date d1;
    Date d2;
    //这里实际上是四个参数
    //d1.Init(&d1,2025, 4, 10);
    d1.Init(2025, 4, 10);
    d1.Print();
    d2.Init(2025, 4, 21);
    d2.Print();
```

```
    return 0;
}
```

小测试：
下述程序的运行结果为( ); A:编译报错 B：运行崩溃 C:正常运行

```
using namespace std;
class A
{
public:
    void Print()
    {
        cout << "haha" << endl;
    }
private:
    int _a;
};

int main() {
    A* p = nullptr;
    p->Print();
    return 0;
}
```

分析：
A* p = nullptr;是创建一个A类型的指针变量p并置为空
p->Print();是不是对空指针解引用呢?
实际上这个p->Print()的实际意义是将p的地址传递过去了，Print函数并不在对象里，同时p本身的地址不为空，所以程序会正常运行。
下述程序的运行结果为( ); A:编译报错 B：运行崩溃 C:正常运行

```
using namespace std;
class A
{
public:
    void Print()
    {
        cout << "haha" << endl;
        cout << _a << endl;
    }
private:
    int _a;
```

```
};

int main() {
    A* p = nullptr;
    p->Print();
    return 0;
}
```

p是 nullptr，调用成员函数 Print()时，编译器把 p作为 this 指针传进去。函数内部访问成员变量 _a时，实际上是在访问this->_a，此时 this为空，相当于试图从内存地址 0（或 0 + _a的偏移量）读取数据，触发了操作系统的内存访问保护机制，导致程序崩溃。
下述程序的运行结果为( ); A:编译报错 B：运行崩溃 C:正常运行

```
using namespace std;
class A
{
public:
    void Print()
    {
        cout << "haha" << endl;
        cout << _a << endl;
    }
private:
    int _a;
};

int main() {
    A* p = nullptr;
    (*p).Print();
    return 0;
}
```

上述程序运行的方式和上一题类似，即将nullptr作为this指针传递进去，函数内部访问成员变量程序会崩溃。但(*p)本身已经是对空指针解引用，是未定义行为。
this指针存在内存的哪个区域( );A
A.栈 B.堆 C.静态区 D.常量区 E.对象里
this指针作为成员函数的隐含参数，和其他函数参数一样，存储在栈上。对象里只存成员变量，不存this。

### 4.C++和C语言的实现Stack对比

面向对象的三大特性：封装、继承、多态。

1. C++中数据和函数都封装到了类里，通过访问限定符进行修饰，不能再随意通过对象直接修改数据，这是C++封装的一种体现也是最重要的变化。这里的封装本质是一种更加严格规范的管理，避免乱访问修改的问题。
2. C++中有一些相对比较方便的语法，比如Init给的缺省参数会方便很多，成员函数每次不需要传对象地址，因为this指针隐含传递了，方便了很多，使用类型不再需要typedef用类名就会很方便。